

Построение ДКА по регулярному выражению на основе алгоритма Глушкова

Постановка

На вход подаётся регулярное выражение. Надо:

1. разобрать его;
2. построить по нему автомат на основе алгоритма Глушкова;
3. получить ДКА;
4. сгенерировать latex код его диаграммы.

Язык регулярных выражений:

$$R ::= 0 \mid 1 \mid a \mid (R \mid R) \mid RR \mid R^*$$

a — символ алфавита, 0 обозначает пустой язык, 1 — пустое слово.

Алгоритм

Регулярное выражение линеаризуется — все вхождения символа получают номер. Например $(a \mid b)^*abb \rightarrow (a_1 \mid b_2)^*a_3b_4b_5$. Дальше вычисляем:

1. $\text{nullable}(R)$ — допускает ли выражение пустое слово;
2. $\text{firstpos}(R)$ — позиции, с которых может начинаться слово;
3. $\text{lastpos}(R)$ — позиции, которыми слово может заканчиваться;
4. $\text{followpos}(i)$ — позиции, которые могут идти сразу после позиции i .

Рекурсивные правила

Для объединения:

$$\text{firstpos}(R_1 \mid R_2) = \text{firstpos}(R_1) \cup \text{firstpos}(R_2)$$

$$\text{lastpos}(R_1 \mid R_2) = \text{lastpos}(R_1) \cup \text{lastpos}(R_2)$$

Для конкатенации:

$$\text{firstpos}(R_1 R_2) = \begin{cases} \text{firstpos}(R_1) \cup \text{firstpos}(R_2), & \text{если } \text{nullable}(R_1) \\ \text{firstpos}(R_1), & \text{иначе} \end{cases}$$

$$\text{lastpos}(R_1 R_2) = \begin{cases} \text{lastpos}(R_1) \cup \text{lastpos}(R_2), & \text{если } \text{nullable}(R_2) \\ \text{lastpos}(R_2), & \text{иначе} \end{cases}$$

И для каждой позиции из $\text{lastpos}(R_1)$ в множество followpos добавляются все позиции из $\text{firstpos}(R_2)$.

Для звезды Клини R^* :

$$\text{nullable}(R^*) = \text{True}$$

$$\text{firstpos}(R^*) = \text{firstpos}(R)$$

$$\text{lastpos}(R^*) = \text{lastpos}(R)$$

И для каждой позиции из $\text{lastpos}(R)$ в followpos добавляются все позиции из $\text{firstpos}(R)$.

Построение ДКА

Сначала по вычисленным множествам задаётся автомат Глушкова:

1. начальное состояние соответствует началу слова;
2. состояния, помеченные последними позициями, принимающие;
3. переходы определяются через firstpos и followpos .

После этого строится ДКА методом подмножеств. Состояние ДКА – множество позиций исходного позиционного автомата.

Начальное состояние ДКА: $\emptyset$

Оно является принимающим $\longleftrightarrow$ исходное регулярное выражение допускает пустое слово.

Результат работы

Программа печатает:

1. список размеченных позиций
2. множества $FirstPos$, $LastPos$, $FollowPos$
3. список состояний ДКА
4. список переходов ДКА.

И программа создаёт $.tex$ - файл с TikZ диаграммой автомата.